Experiment No. 9 (Software) **Date: 09.10.2022**

Design of 4-bit Parallel Adder and 2-bit Multiplier

Aim: To implement Parallel Adder and 2-bit Multiplier using logic gates in Verilog Programming.

Software Required: ModelSim

Procedure:

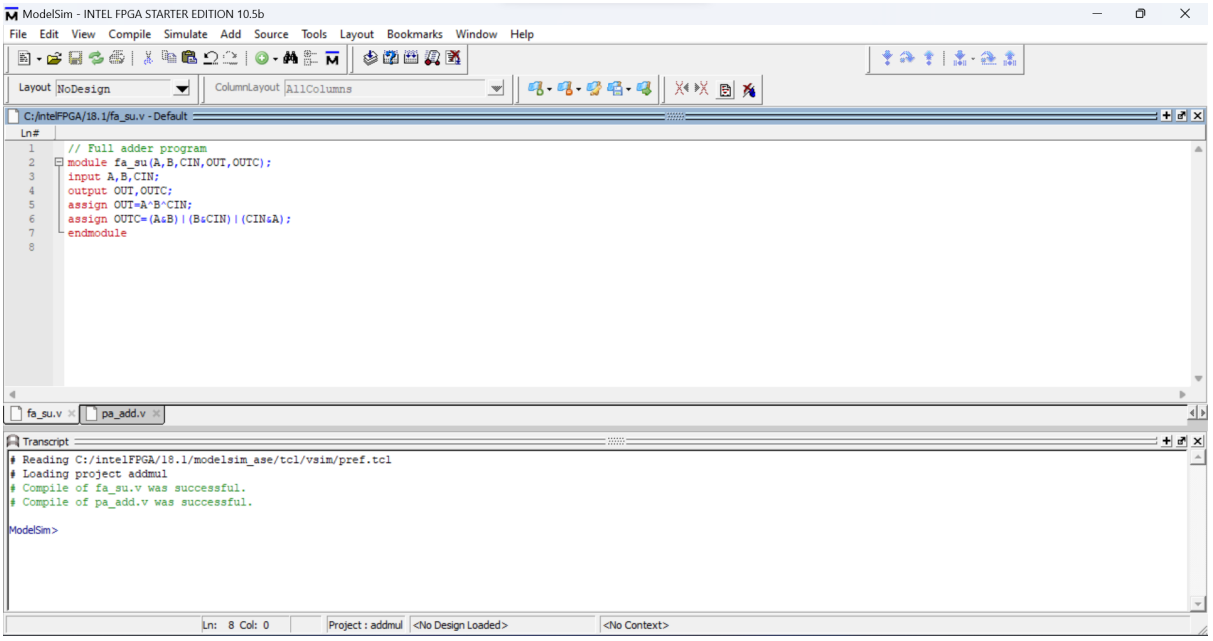
1. Open ModelSim.
2. Create New Project, then create a new file by selecting the verilog in the drop-down.
3. Type the code, then compile it.
4. Type the test bench, compile and simulate it.
5. Click add wave in the window and then run it.

Verilog Code:

<u>Parallel Adder</u>

Full Adder:

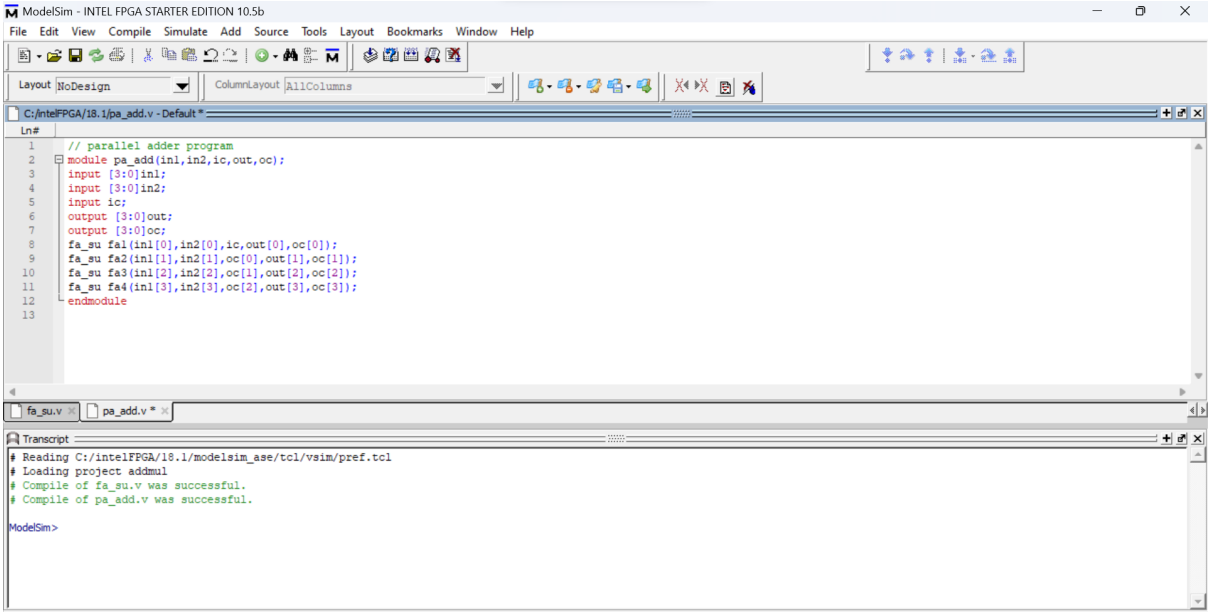
```
// Full Adder Program
module fa_su(A,B,CIN,OUT,OUTC);
input A,B,CIN;
output OUT,OUTC;
assign OUT=A^B^CIN;
assign OUTC=(A&B)|(B&CIN)|(CIN&A);
endmodule
```



Parallel Adder Program:

```
// parallel adder program
module pa_add(in1,in2,ic,out,oc);
input [3:0]in1;
input [3:0]in2;
```

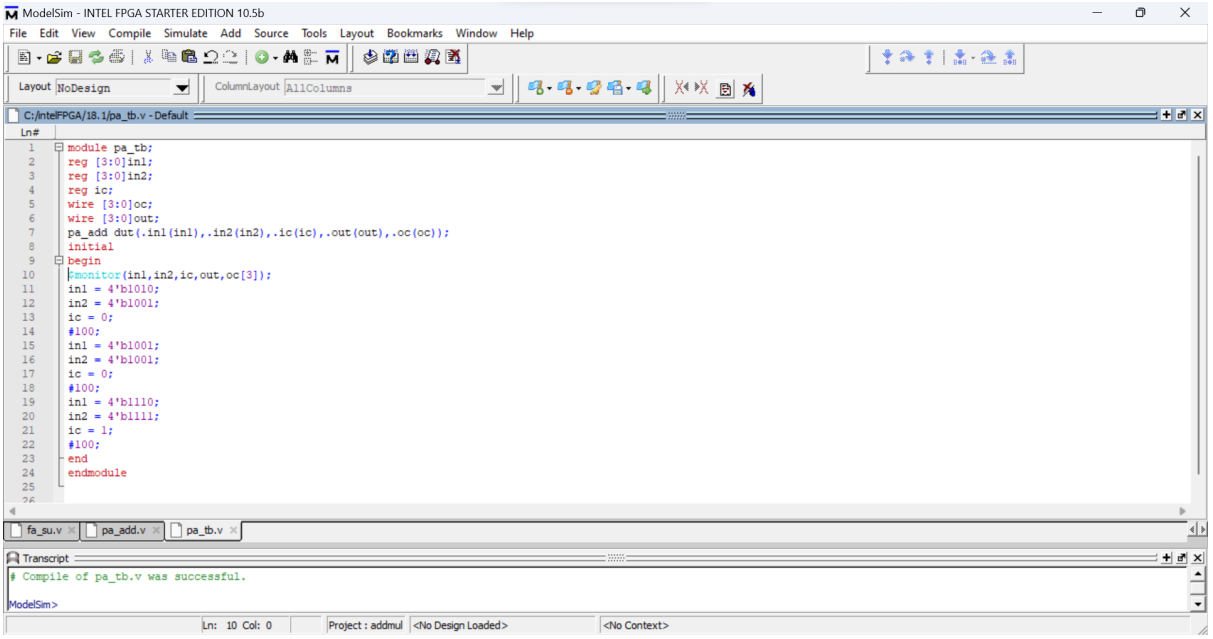
```
input ic;
output [3:0]out;
output [3:0]oc;
fa_su fa1(in1[0],in2[0],ic,out[0],oc[0]);
fa_su fa2(in1[1],in2[1],oc[0],out[1],oc[1]);
fa_su fa3(in1[2],in2[2],oc[1],out[2],oc[2]);
fa_su fa4(in1[3],in2[3],oc[2],out[3],oc[3]);
endmodule
```



Test Bench:

```
module pa_tb;
reg [3:0]in1;
reg [3:0]in2;
reg ic;
wire [3:0]oc;
wire [3:0]out;
pa_add dut(.in1(in1),.in2(in2),.ic(ic),.out(out),.oc(oc));
initial
begin
$monitor(in1,in2,ic,out,oc[3]);
in1 = 4'b1010;
in2 = 4'b1001;
ic = 0;
#100;
in1 = 4'b1001;
in2 = 4'b1001;
ic = 0;
#100;
in1 = 4'b1110;
in2 = 4'b1111;
ic = 1;
#100;
```

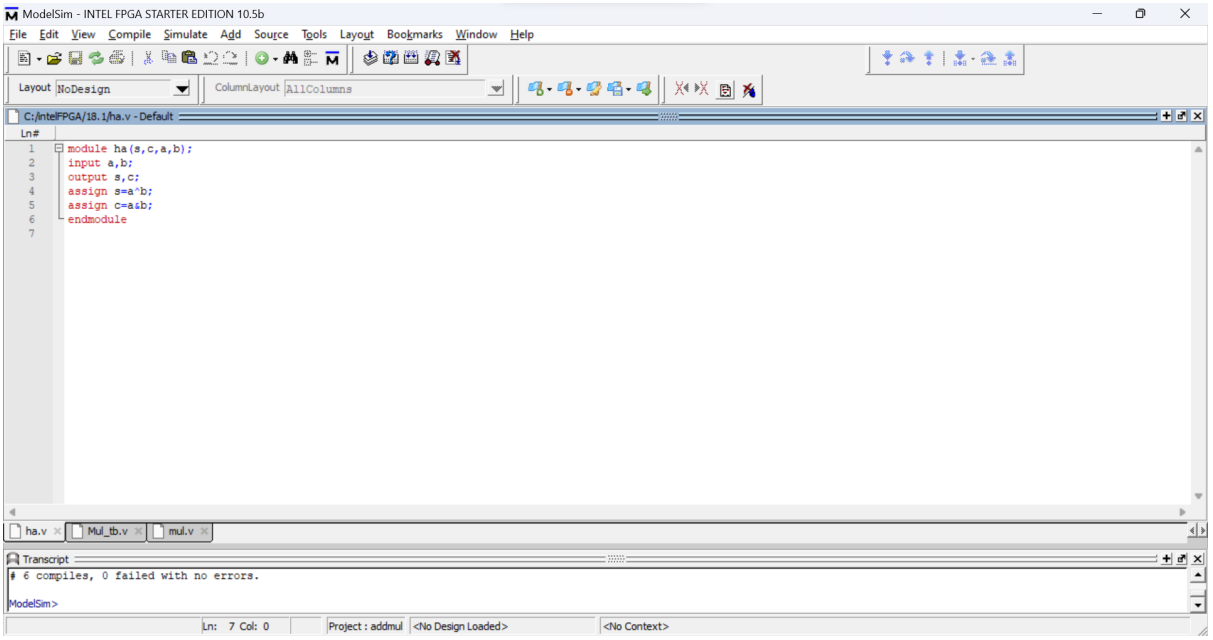
end
endmodule



2- bit Multiplier

Half Adder:

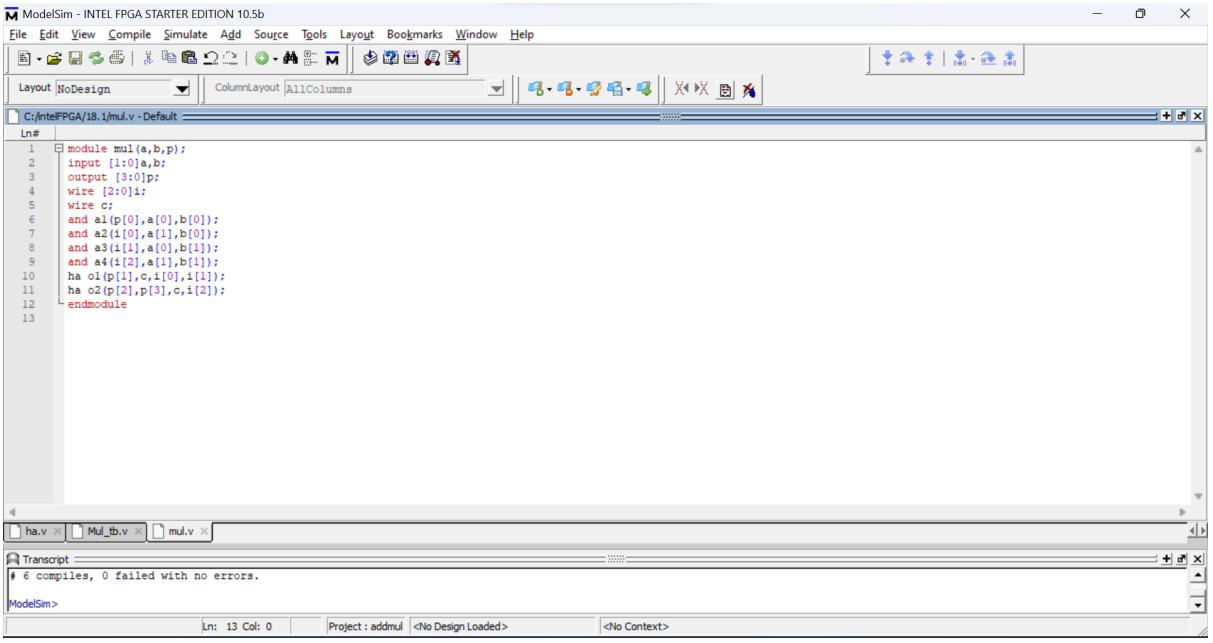
module ha(s,c,a,b);
input a,b;
output s,c;
assign s=a^b;
assign c=a&b;
endmodule



Multiplier:

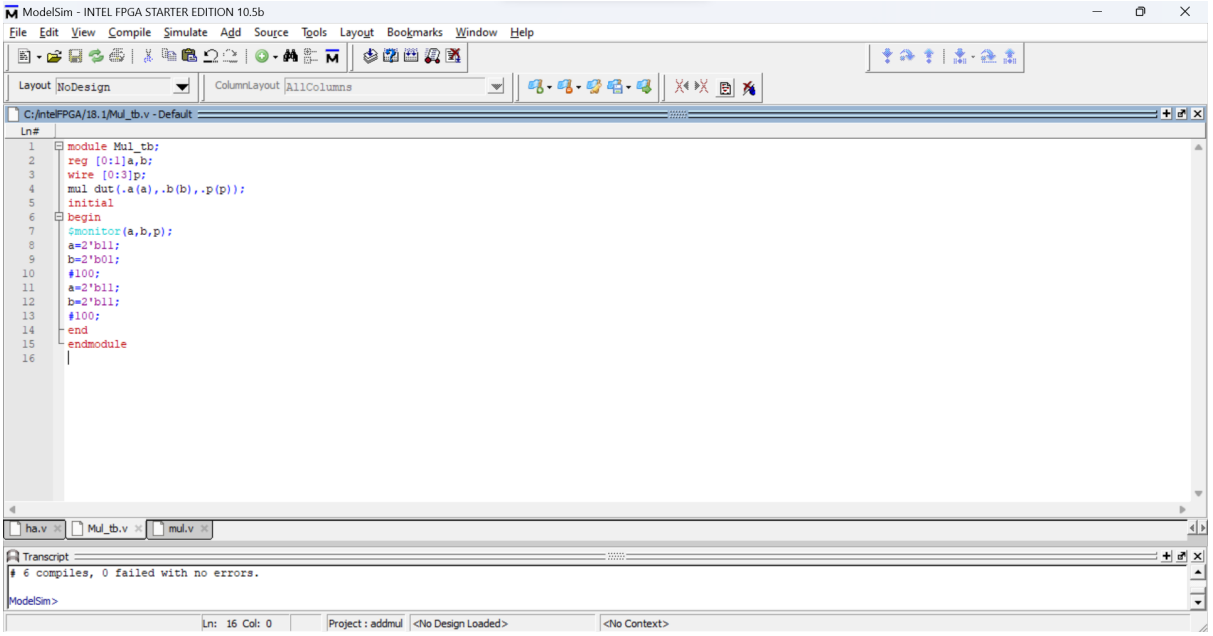
module mul(a,b,p);
input [1:0]a,b;
output [3:0]p;

```
wire [2:0]i;
wire c;
and a1(p[0],a[0],b[0]);
and a2(i[0],a[1],b[0]);
and a3(i[1],a[0],b[1]);
and a4(i[2],a[1],b[1]);
ha o1(p[1],c,i[0],i[1]);
ha o2(p[2],p[3],c,i[2]);
endmodule
```



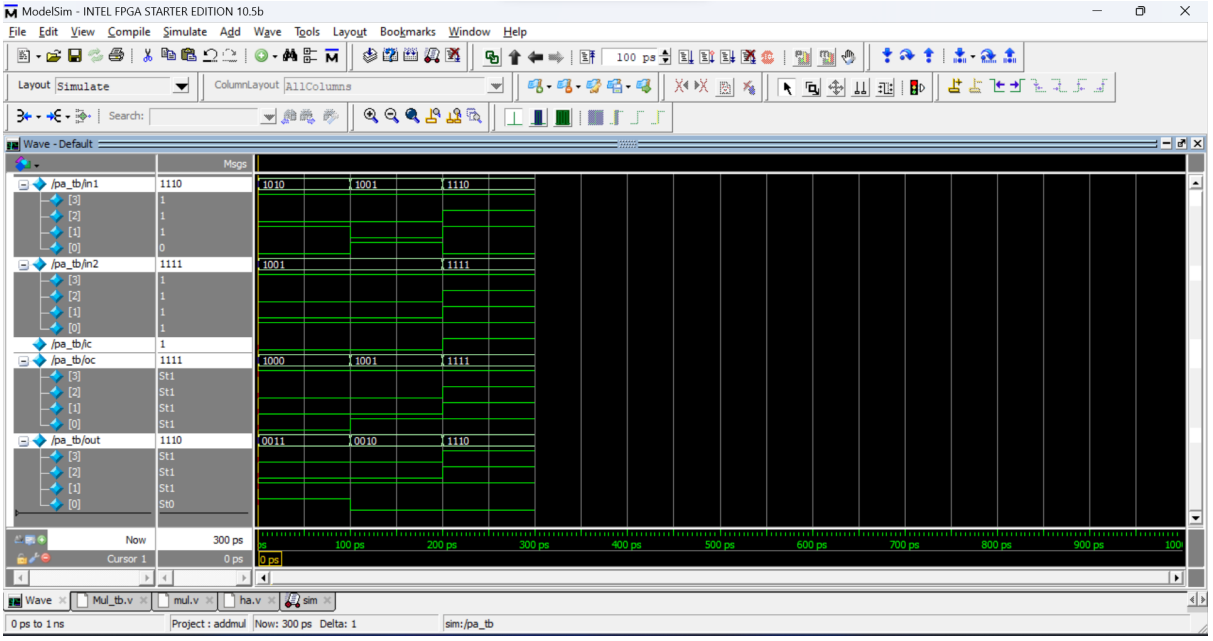
Test Bench:

```
module Mul_tb;
reg [0:1]a,b;
wire [0:3]p;
mul dut(.a(a),.b(b),.p(p));
initial
begin
$monitor(a,b,p);
a=2'b11;
b=2'b01;
#100;
a=2'b11;
b=2'b11;
#100;
end
endmodule
```

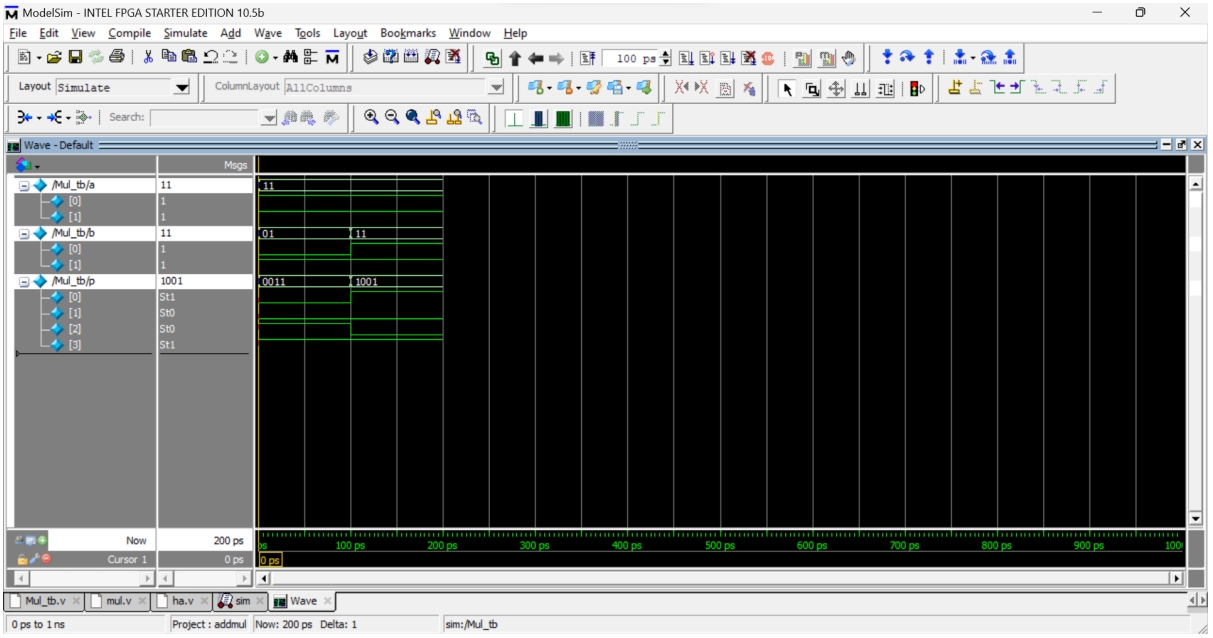


Output:

Parallel Adder:



Multiplier:



Result/ Inference:

From this experiment, the Parallel adder and multiplier circuits are verified using data-flow modelling and behavioural modelling using verilog programming.